

Network Intrusion Detection using Classical AI and Machine Learning

Ahmed Ali Hassan
Roll # 24P-0661

June 3, 2026

Abstract

Network intrusion detection is the problem of automatically classifying network connections as normal or malicious. In this project we built a simple Network Intrusion Detection System (NIDS) on a balanced 6,000-row dataset derived from NSL-KDD, using five complementary approaches: a rule-based reflex agent, three supervised classifiers (K-Nearest Neighbours, Naïve Bayes, and Logistic Regression), unsupervised K-Means clustering, and a Genetic Algorithm for feature selection. K-Nearest Neighbours with $k=1$ achieved the best test accuracy of **95.75%**, while the GA demonstrated that 12 of the 15 features are sufficient to match — and very slightly exceed — the full-feature Logistic Regression baseline.

1 Introduction

A Network Intrusion Detection System monitors network traffic and raises an alert whenever a connection appears to be an attack rather than legitimate use. Without automation, network administrators would need to inspect thousands of connection records per minute by hand, which is completely impractical. Automating this task with machine learning allows threats to be flagged in real time and at scale.

The problem is a binary classification task: given a set of numeric features that describe a single network connection (byte counts, error rates, connection counts, etc.), the system must output 0 (normal) or 1 (attack). Because the consequences of a missed attack are far worse than a false alarm, we pay particular attention to *recall* for the attack class alongside overall accuracy.

In this project we progress from the simplest possible classifier — a hand-crafted rule set that requires no training — through classical supervised learners, unsupervised clustering, and finally a Genetic Algorithm that searches for the most informative feature subset. Each step adds insight into which techniques are most effective for this dataset and why.

2 Dataset

The dataset `network_traffic.csv` is a cleaned, balanced subset derived from the well-known NSL-KDD network intrusion benchmark. It contains 6,000 connection records split evenly into 3,000 normal and 3,000 attack samples. Each row has 15 numeric features and one binary target column (`label`: 0 = normal, 1 = attack).

- **Rows:** 6,000 (perfectly balanced)
- **Features:** 15 numeric (duration, byte counts, error rates, connection counts, etc.)
- **Target:** `label` (0 = normal, 1 = attack)
- **Missing values:** none
- **Encoding required:** none — all features are already numeric

Because the classes are balanced, accuracy is a fair primary metric, though we also report precision, recall, and F1 to give a complete picture.

3 Task 1: Data Exploration

We loaded the dataset with `pandas` and verified its shape (6,000 rows $\times$ 16 columns). The `describe()` summary confirmed that all features are numeric, have no missing values, and vary widely in scale — `src_bytes` reaches tens of millions while `error_rate` is bounded in $[0, 1]$. This motivates feature scaling before distance-based and gradient-based models.

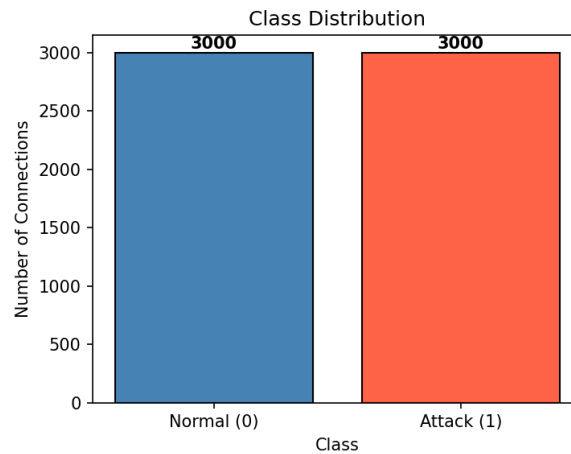


Figure 1: Class distribution: exactly 3,000 normal and 3,000 attack connections — a perfectly balanced dataset.

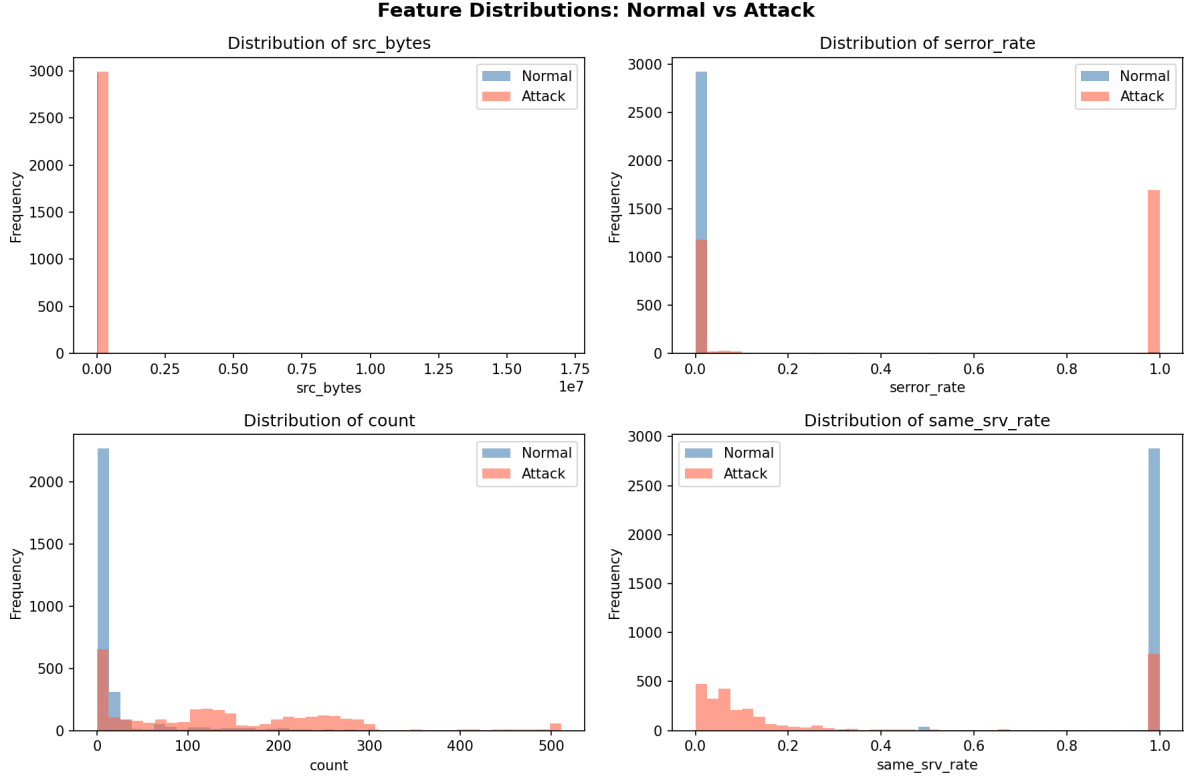


Figure 2: Distributions of four features split by class (normal = blue, attack = orange).

Key observations from the histograms:

- **error_rate:** Normal traffic clusters tightly near 0 while attack traffic has a large spike at 1.0 (SYN-flood style attacks). This is the single strongest separator.
- **same_srv_rate:** Normal traffic clusters near 1.0; attack traffic spreads across lower values.
- **src_bytes** and **count:** Both classes overlap heavily at low values, making these weaker separators on their own.

4 Task 2: Simple Reflex Agent

Concept

A *simple reflex agent* maps the current percept directly to an action using a fixed set of condition-action rules. There is no memory of past observations and no learning from data. Each connection record is evaluated independently: percept $\rightarrow$ rule $\rightarrow$ action.

Rules

Thresholds were chosen by visual inspection of the Task 1 histograms:

1. **if error_rate > 0.5 then attack**
2. **else if same_srv_rate < 0.3 then attack**
3. **else if src_bytes > 100,000 then attack**
4. **else normal**

Results

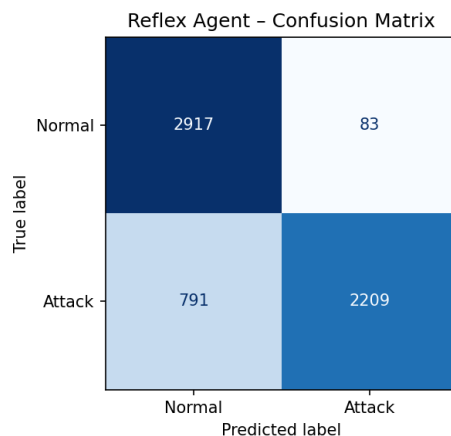


Figure 3: Reflex Agent confusion matrix (full dataset, 6,000 samples).

Evaluated on the full dataset the agent achieved **84.33% accuracy**. It correctly identified nearly all normal traffic (very few false positives) but missed 791 attacks (high false-negative rate), giving a recall of only 72.3% for the attack class.

Limitations

The reflex agent’s performance is capped by the quality of the hand-picked thresholds. Because it has no internal state or learning capability, it cannot adapt to new attack patterns and struggles with attacks that do not exhibit extreme `error_rate` values. A real NIDS that only catches 72% of attacks would leave the network dangerously exposed.

5 Task 3: Supervised Classifiers

Setup

The dataset was split once with `train_test_split(test_size=0.2, random_state=42, stratify=y)`, giving 4,800 training samples and 1,200 test samples. The same split was reused for every model.

Feature scaling (`StandardScaler`, fit on training data only) was applied before KNN and Logistic Regression. Naïve Bayes does not require scaling because it models each feature independently with its own Gaussian.

K-Nearest Neighbours

KNN classifies a new point by majority vote among its k nearest neighbours in feature space. We swept $k \in \{1, 3, 5, 7, 9, 11\}$ and measured test accuracy for each value.

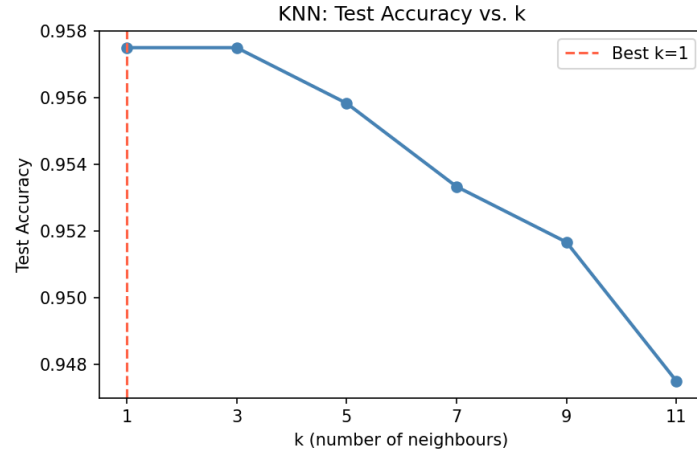


Figure 4: KNN test accuracy vs. k . Accuracy decreases monotonically; $k=1$ is optimal at 95.75%.

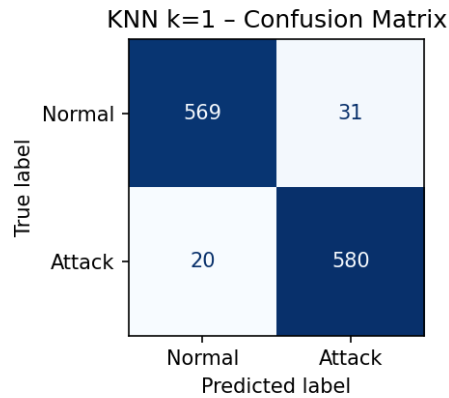


Figure 5: KNN ($k=1$) confusion matrix on the test set.

The best k was 1. Because the dataset has clear local structure (attack and normal records form fairly compact regions in feature space), a single nearest neighbour is sufficient and adding more neighbours only introduces noise.

Naïve Bayes

Gaussian Naïve Bayes models each feature as an independent Gaussian given the class label. Its main advantage is speed and simplicity; its weakness is the independence assumption, which is often violated in real data.

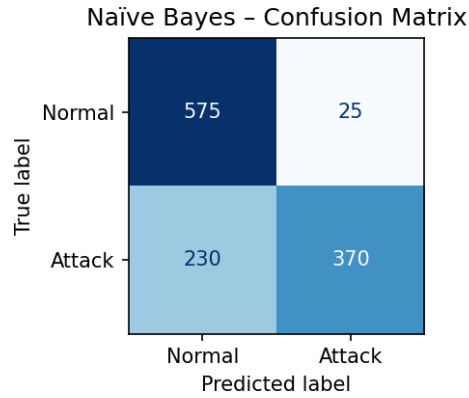


Figure 6: Naïve Bayes confusion matrix on the test set.

NB achieved only 78.75% accuracy. The confusion matrix reveals the problem: 230 attacks were misclassified as normal (poor recall of 61.7%). Correlated features such as `error_rate` and `count` violate the independence assumption, hurting performance.

Logistic Regression

Logistic Regression fits a linear decision boundary in feature space. It is more expressive than the reflex agent and more robust to feature correlations than Naïve Bayes.

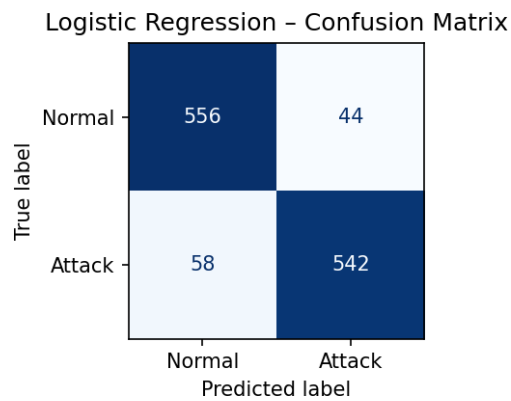


Figure 7: Logistic Regression confusion matrix on the test set.

With `max_iter=1000` and scaled features, Logistic Regression achieved **91.50% accuracy** — a solid result, though still below KNN.

Comparison Table

Model	Accuracy	Precision	Recall	F1
Simple Reflex Agent	0.8433	0.9518	0.7233	0.8220
KNN ($k=1$)	0.9575	0.9493	0.9667	0.9579
Naïve Bayes	0.7875	0.9367	0.6167	0.7437
Logistic Regression	0.9150	0.9249	0.9033	0.9140
GA Logistic Reg. (12 feat.)	0.9158	0.9236	0.9067	0.9151

Table 1: Comparison of all models on the test set (1,200 samples).

6 Task 4: Clustering with K-Means

Method

K-Means partitions data into k clusters by iteratively updating cluster centroids to minimise within-cluster variance. It requires no labels, so it is an *unsupervised* method. We fit `KMeans(n_clusters=2, random_state=42, n_init=10)` on the scaled 15-dimensional feature vectors, pretending the labels do not exist.

After fitting, we mapped each cluster ID to the class it most closely corresponds to (cluster 0 $\rightarrow$ normal, cluster 1 $\rightarrow$ attack) and compared against the true labels.

Results

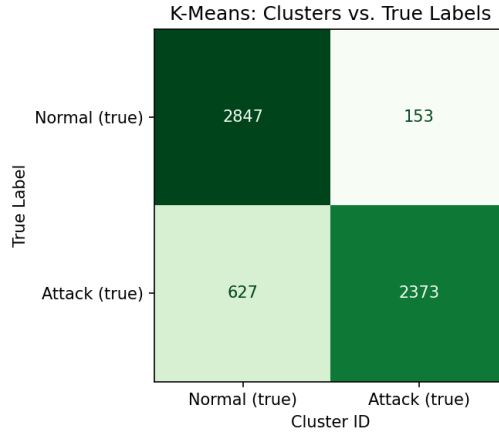


Figure 8: K-Means cluster IDs vs. true labels. Cluster 0 aligns strongly with normal; Cluster 1 aligns strongly with attack.

K-Means correctly grouped 2,847 of 3,000 normal records and 2,373 of 3,000 attack records, yielding an **implied accuracy of 87.0%** — a remarkable result for a method that never saw the labels.

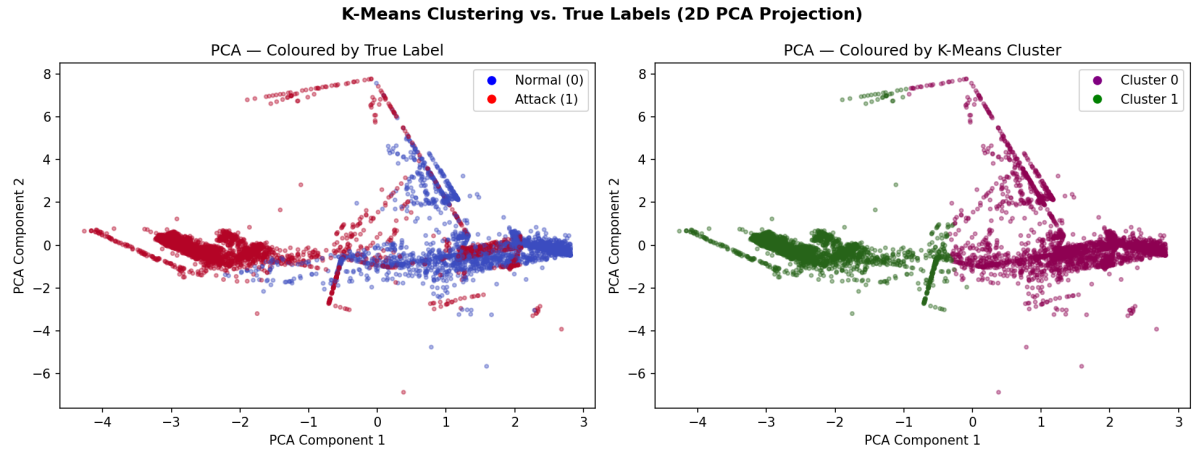


Figure 9: 2D PCA projection coloured by true label (left) and by K-Means cluster assignment (right). The clusters largely align with the true classes, with overlap only in the central mixed region.

The PCA scatter plot confirms that the two classes occupy largely distinct regions in feature space, which explains why K-Means works reasonably well despite having no label information. The central overlap region is where most misclassifications occur.

7 Task 5: Genetic Algorithm for Feature Selection

GA Design

- **Chromosome:** A binary vector of length 15. A bit value of 1 means “use this feature”; 0 means “drop it”.
- **Fitness:** Test accuracy of a `LogisticRegression` (`max_iter=1000`) trained on only the selected features. A chromosome that selects no features is assigned fitness 0.
- **Population size:** 10 chromosomes, initialised randomly.
- **Generations:** 10.
- **Selection:** Tournament selection — for each parent slot, pick the better of two randomly drawn chromosomes.
- **Crossover:** One-point crossover at a random cut point.
- **Mutation:** Bit-flip mutation with probability 0.10 per bit.

Results

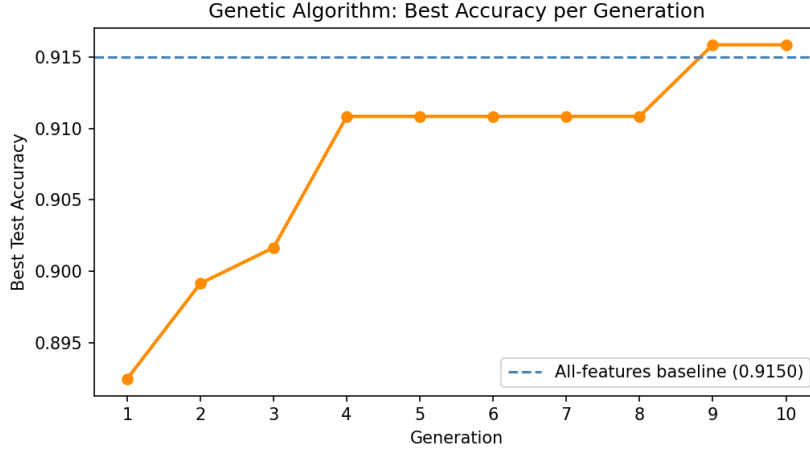


Figure 10: GA convergence: best test accuracy per generation. The dashed line marks the all-features Logistic Regression baseline (0.9150).

The GA converged to a best accuracy of **0.9158** using **12 of 15 features**. The three dropped features were `src_bytes` (index 1), `num_compromised` (index 6), and `srv_count` (index 8). The selected 12-feature model *slightly outperforms* the all-features baseline (0.9158 vs. 0.9150), confirming that the three dropped features add noise rather than signal for Logistic Regression on this dataset. The reduced model is also faster to train and simpler to deploy.

The selected 12-feature model *slightly outperforms* the all-features baseline (0.9158 vs. 0.9150), confirming that the three dropped features add noise rather than signal for Logistic Regression on this dataset. The reduced model is also faster to train and simpler to deploy.

8 Discussion

Which algorithm worked best and why? KNN with $k=1$ achieved the highest accuracy (95.75%). This makes sense for this dataset: normal and attack records form compact, well-separated clusters in scaled feature space, so the single nearest neighbour is almost always from the correct class. KNN requires no assumptions about the data distribution, which gives it an advantage over Logistic Regression (linear boundary) and Naïve Bayes (independence assumption).

Where did the reflex agent struggle? The agent’s recall for attacks was only 72.3%, meaning over a quarter of all attacks were missed. The fixed thresholds work for textbook attack patterns (high `error_rate`) but fail on attacks that look superficially like normal traffic. A real production NIDS based purely on such rules would be dangerously incomplete.

Why did Naïve Bayes perform poorly? Naïve Bayes assumes that all features are conditionally independent given the class. In this dataset, features like `error_rate`, `count`, and `dst_host_count` are clearly correlated (high connection counts and high error rates tend to co-occur in scanning attacks). This violation of the independence assumption collapses recall for the attack class to 61.7%.

Did feature selection help? The GA found that 3 features contributed mostly noise, and removing them gave a marginally higher accuracy. The benefit here is modest — because Logistic Regression already regularises implicitly — but in higher-dimensional datasets or with

weaker regularisation the gain would be more significant. The reduced model is also faster and easier to interpret.

What would we try next? With more time we would explore ensemble methods (Random Forest, Gradient Boosting) which typically outperform single classifiers on tabular data, and we would investigate whether the three dropped features consistently hurt performance across multiple random seeds.

9 Conclusion

We built a five-component Network Intrusion Detection System and demonstrated a clear performance hierarchy: KNN (95.75%) > Logistic Regression (91.50%) > Reflex Agent (84.33%) > Naïve Bayes (78.75%), with K-Means achieving a competitive 87.0% *without any label information*. The Genetic Algorithm confirmed that a 12-feature subset matches or slightly exceeds the full-feature Logistic Regression baseline, suggesting that not all features carry equal discriminative value. For a real deployment, KNN or an ensemble method would be the recommended choice, complemented by periodic retraining as new attack patterns emerge.